# Graph databases

**Vassányi István**
**Pannonian University , Veszprém**

# The basic idea

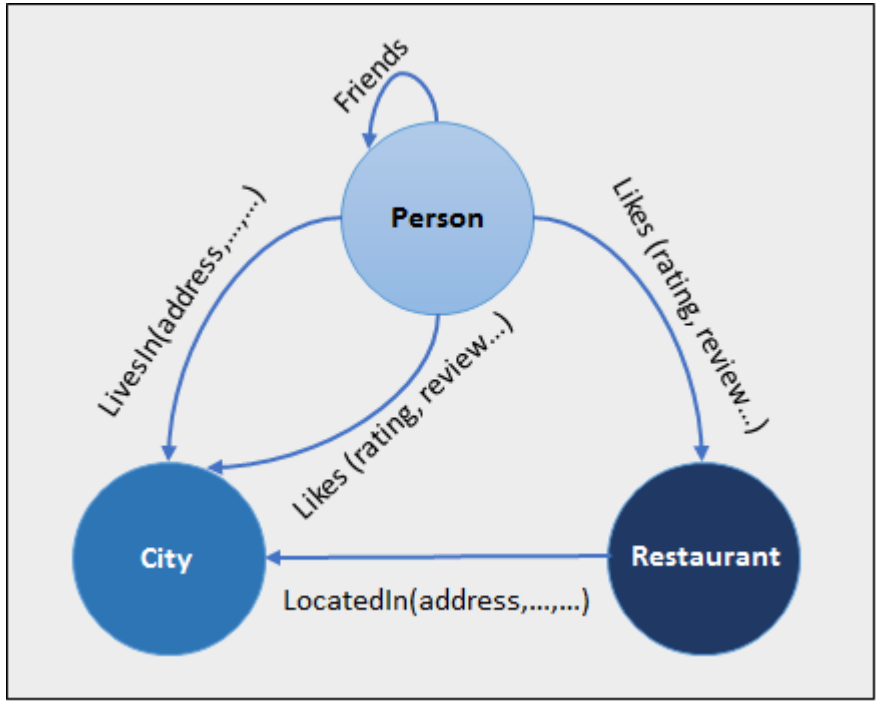
- Database design starts with the domain model
  - UML class diagram: association, aggregation , composition, inheritance
  - EER diagram: relationships of different cardinalities , specialization/generalization
- Relational logical data model:
  - 3 types of relationships: 1: 1 , 1 :N, N:M for all the above cases (simplification)
  - 1: 1 , 1 :N -> foreign key, N:M -> linking table
- Graph database: network data model
- THE DATABASE HAS ONLY EDGES AND NODES
  - Edges: directed or undirected, attributes
  - Nodes: identifiers, attributes

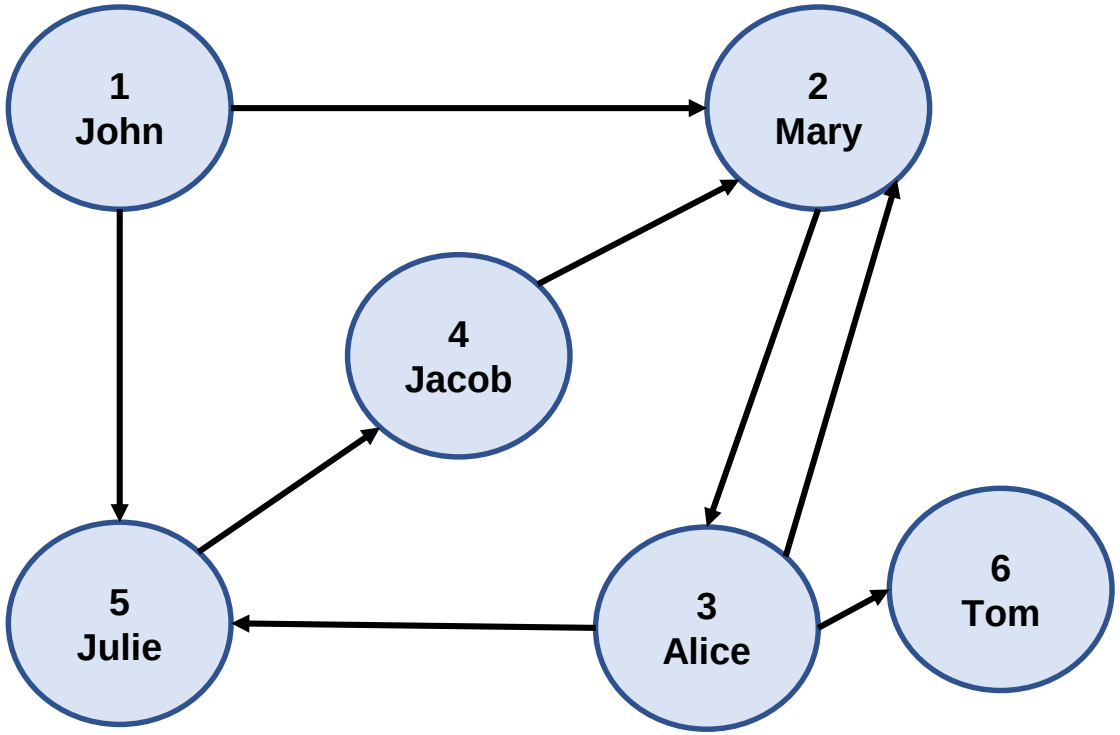
# Graph database as a relational extension

- Create tables: as node , as edge
- Automatically generated identifiers
  - $node _id, $ edge _id , $ from _id , $ to
- Type constraints can be placed on edges
- No undirected edge
- Can be mapped to the relational schema
- Query with search pattern ( match pattern )
  - Intuitive, easy to understand, popular

DEMO

# The demo database

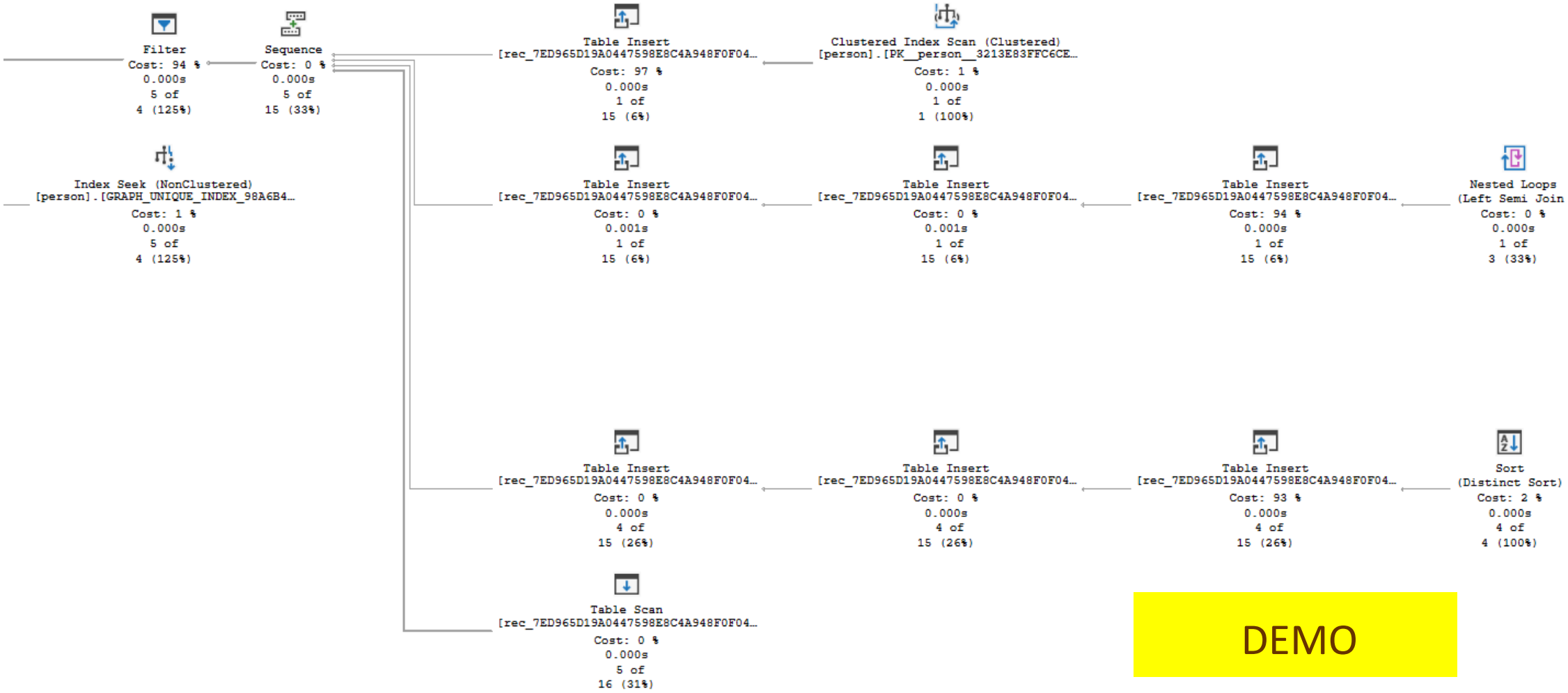


# Shortest path execution plan

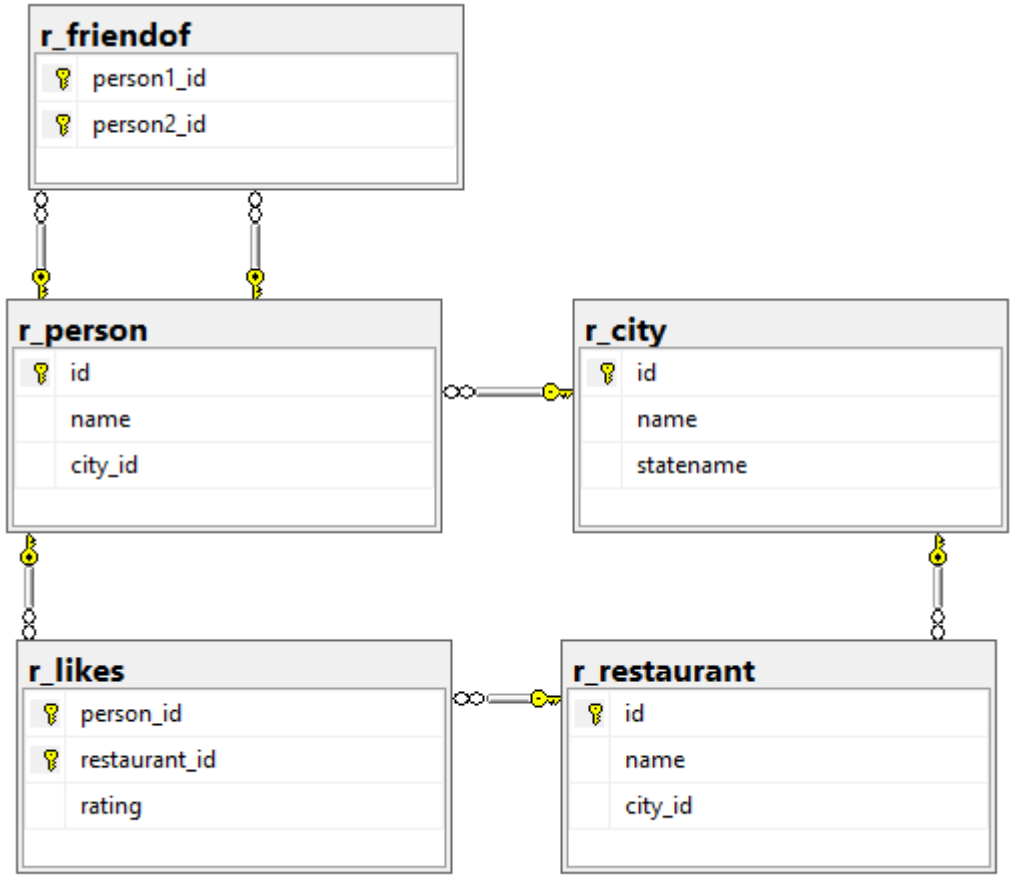
Query 1: Query cost (relative to the batch): 100%

```
SELECT name, paths, depth FROM (SELECT p1.name, p1.name + '->' + string_agg(p2.name, '->') WITHIN GROUP ( GRAPH PATH) AS paths, last_value(p2.Name) WITHIN GROUP ( GRAPH P…
```




DEMO

# Same in relational model




DEMO

# Relational->graph migration

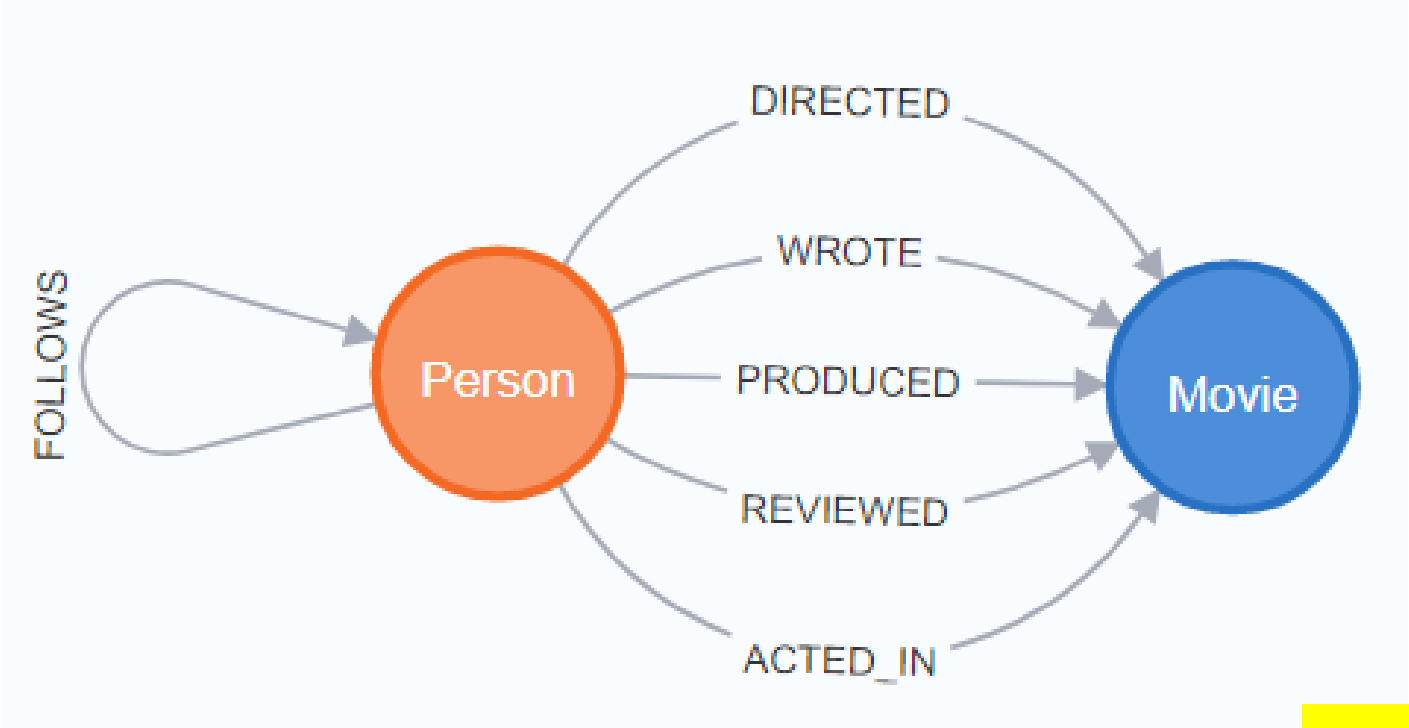
- Every relationship will become an edge
- We can only create edges after nodes, by querying the internal identifiers

DEMO

# Neo4j, a Java-based native graph database

- Efficient binary storage, separate for nodes, labels, relationships (=edges), and attributes
- A node can have multiple labels, NO node type
- A relationship can only have one type
- "optional" scheme, constraints can be created
- Query language: Cypher , based on pattern matching

# Demo graph database




DEMO